@app — refine/suggest

Appears in: **DL-06**

app.py:215-224, 227-308

"…the /refine endpoint, ReAct routing in assistant.py, SSE progress on /generate …" (and "The full output state travels as JSON in every /refine request, so every call is independently auditable.")

@app — refine/suggest

app.py:215-224

```python
@app.route("/suggest", methods=["POST"])
def suggest():
    data = request.get_json(force=True, silent=True)
    if not data or "output" not in data:
        return jsonify({"message": "Could not analyze campaign output."}), 400
    try:
        message = assistant.suggest(data["output"])
        return jsonify({"message": message})
    except Exception as e:
        return jsonify({"message": f"Suggestion service unavailable: {e}"}), 200
```

…

app.py:227-308

```python
@app.route("/refine", methods=["POST"])
def refine():
    data = request.get_json(force=True, silent=True)
    if not data:
        return jsonify({"error": "Invalid request body"}), 400

    current_output = data.get("output_state", {})
    user_message   = data.get("user_message", "").strip()
    tone_val       = int(data.get("tone", 3))
    age_val        = int(data.get("age", 35))
    formality_val  = int(data.get("formality", 3))
    chat_history   = data.get("chat_history", [])

    routing = assistant.get_routing(
        current_output, user_message, tone_val, age_val, formality_val, chat_history
    )
    agents_to_run = routing["agents_to_run"]

    params = assistant.map_sliders_to_params(tone_val, age_val, formality_val)
    tone_str        = params["tone"]
    age_hint        = params["age_hint"]
    formality_instr = params["formality_instruction"]

    updated_output = dict(current_output)
    updated_output["meta"] = dict(current_output.get("meta", {}))
    updated_output["meta"]["tone"] = tone_str

    description = current_output.get("meta", {}).get("description", "")
    vision_data = current_output.get("vision", {})
    color_data  = current_output.get("colors", {})

    doc_context = []
    if agents_to_run:
        rag_query = description + " " + " ".join(vision_data.get("product_tags", []))
        doc_context = rag.retrieve(rag_query)
        updated_output["rag_chunks_used"] = len(doc_context)

    audience_data = current_output.get("audience", {})

    if "audience" in agents_to_run:
        augmented_vision = dict(vision_data)
        augmented_vision["target_signals"] = list(
            vision_data.get("target_signals", [])
        ) + [f"target age group: {age_hint}"]
        audience_data = audience.segment(description, augmented_vision, doc_context=doc_context)
        updated_output["audience"] = audience_data

    ads_dict = current_output.get("ads", {})

    if "copywriter" in agents_to_run:
        effective_tone = f"{tone_str}. Writing style: {formality_instr}"
        ads_dict = copywriter.generate(
            description, vision_data, color_data,
            effective_tone, audience_data, doc_context=doc_context
        )

    guardrails_result = {
        "clean_copy": ads_dict,
        "flags": current_output.get("compliance_flags", []),
    }

    if "guardrails" in agents_to_run:
        guardrails_result = guardrails.check(ads_dict)
        updated_output["ads"] = guardrails_result["clean_copy"]
        updated_output["compliance_flags"] = guardrails_result["flags"]

    if "cta_optimizer" in agents_to_run:
        cta_result = cta_optimizer.optimize(
            guardrails_result["clean_copy"], tone_str, audience_data, doc_context=doc_context
        )
        updated_output["cta_analysis"] = cta_result

    assistant_message = _build_assistant_reply(
        routing["reasoning"], agents_to_run, updated_output
    )

    return jsonify({
        "output": updated_output,
        "routing_label": routing["routing_label"],
        "reasoning": routing["reasoning"],
        "assistant_message": assistant_message,
    })
```

What it shows: The stateless refine endpoint: it reads the full output state from the request, runs only the agents the router chose, and returns the updated state + routing label. `/suggest` powers the proactive review.

Source: app.py:215-224, 227-308 · image rendered by build_snippets.py